
Rapport de Projet

**Mise en Place d'un Centre
Opérationnel de Sécurité (SOC)
Basé sur des Outils Open Source**

pfSense • Wazuh • Suricata • TheHive • Shuffle

Filière :	ING4-J — SSIR-H
Réalisé par :	Hamdi Maïssa Reguigui Aymen
Encadrée par :	Sahar Ben Yaala

Année universitaire : 2025 – 2026

Résumé

Ce projet de fin d'études porte sur la conception et la mise en œuvre d'un Centre Opérationnel de Sécurité (SOC) entièrement basé sur des outils open source dans un environnement virtualisé. L'objectif principal est de déployer une infrastructure de surveillance, de détection des intrusions, de gestion des incidents et d'automatisation de la réponse, couvrant ainsi le cycle complet du SOC : *Monitor* → *Detect* → *Analyse* → *Respond*.

L'infrastructure repose sur **pfSense** comme pare-feu et routeur principal intégrant le moteur de détection d'intrusions **Suricata**, sur **Wazuh** comme plateforme SIEM/EDR pour la collecte et l'analyse des journaux, sur **TheHive** comme système de gestion des cas d'incidents, et sur **Shuffle** pour l'automatisation des workflows SOAR.

Des scénarios d'attaque réels ont été simulés depuis une machine Kali Linux (scan Nmap, brute-force SSH) pour valider les capacités de détection et de réponse automatisée de l'ensemble de la chaîne.

Mots-clés : SOC, SIEM, SOAR, pfSense, Wazuh, Suricata, TheHive, Shuffle, Cybersécurité, Open Source.

Abstract

This final year project focuses on the design and deployment of a Security Operations Center (SOC) entirely based on open source tools in a virtualized environment. The main objective is to build a complete infrastructure for monitoring, intrusion detection, incident management, and automated response, covering the full SOC cycle : *Monitor* $\rightarrow$ *Detect* $\rightarrow$ *Analyse* $\rightarrow$ *Respond*.

The infrastructure relies on **pfSense** as the main firewall and router integrating the **Suricata** IDS/IPS engine, on **Wazuh** as the SIEM/EDR platform for log collection and analysis, on **TheHive** as the incident case management system, and on **Shuffle** for SOAR workflow automation.

Real attack scenarios were simulated from a Kali Linux machine (Nmap scan, SSH brute-force) to validate the detection and automated response capabilities of the entire chain.

Keywords : SOC, SIEM, SOAR, pfSense, Wazuh, Suricata, TheHive, Shuffle, Cybersecurity, Open Source.

Table des matières

Résumé	1
Abstract	2
Table des matières	3
Table des figures	5
Liste des tableaux	7
1 Introduction Générale	8
1.1 Contexte et Motivation	8
1.2 Problématique	9
1.3 Objectifs du Projet	9
1.4 Structure du Rapport	9
2 État de l'Art et Cadre Théorique	10
2.1 Le Security Operations Center (SOC)	10
2.1.1 Définition et Rôle	10
2.1.2 Composantes Technologiques d'un SOC	10
2.2 Présentation des Outils Utilisés	11
2.2.1 pfSense	11
2.2.2 Suricata	11
2.2.3 Wazuh	12
2.2.4 TheHive	12
2.2.5 Shuffle	12
2.2.6 DVWA	12
2.2.7 Kali Linux	12
2.3 Comparaison avec des Solutions Propriétaires	12
3 Architecture de la Solution	14
3.1 Vue Globale de l'Architecture	14
3.2 Flux de Données du SOC	15
3.3 Adressage IP et Segmentation Réseau	15
3.4 Matériel et Logiciels	16

4	Déploiement et Configuration	17
4.1	Configuration de pfSense	17
4.1.1	Interfaces Réseau	17
4.1.2	Règles de Pare-feu	17
4.1.3	Configuration de Suricata sur pfSense	19
4.2	Déploiement de Wazuh	19
4.2.1	Architecture Wazuh	19
4.2.2	Tableau de Bord Wazuh	20
4.2.3	Agents Actifs	20
4.2.4	Agent Windows et DMZ	21
4.2.5	Vues Globales du Tableau de Bord	21
4.3	Déploiement de DVWA	22
4.4	Configuration du Workflow Shuffle	22
5	Tests, Résultats et Validation	23
5.1	Scénarios d’Attaque Simulés	23
5.1.1	Scénario 1 : Scan Nmap (Reconnaissance)	23
5.1.2	Scénario 2 : Brute-Force SSH	23
5.2	Résultats de Détection	24
5.2.1	Alertes Suricata sur pfSense	24
5.2.2	Événements Wazuh – Brute-Force Détecté	25
5.2.3	Événements Suricata dans Wazuh	26
5.2.4	Tableaux de Bord de Sécurité par Zone	26
5.3	Gestion des Incidents avec TheHive	27
5.3.1	Alertes Reçues dans TheHive	27
5.3.2	Cas d’Incident Créé	27
5.3.3	Marquage et Clôture du Cas	28
5.4	Synthèse des Résultats	28
5.5	Analyse des Performances	29
	Conclusion Générale et Perspectives	30
	Références Bibliographiques	31
A	Récapitulatif des Commandes Utilisées	32
A.1	Commandes Kali Linux – Tests d’Intrusion	32
A.2	Configuration Wazuh – Intégration Shuffle	32

Table des figures

1.1	Cycle opérationnel d'un SOC : Monitor – Detect – Analyse – Respond	8
3.1	Architecture réseau globale du SOC (LAN, DMZ, WAN)	14
3.2	Flux de données entre les composants du SOC (Wazuh, Shuffle, TheHive, Elasticsearch)	15
4.1	Console pfSense 2.8.1 – Configuration des interfaces WAN, LAN et DMZ	17
4.2	Règles de pare-feu sur l'interface WAN – Filtrage du trafic entrant depuis Internet	18
4.3	Règles de pare-feu sur l'interface LAN – Contrôle des accès depuis le réseau interne	18
4.4	Règles de pare-feu sur l'interface DMZ – Isolation de la zone démilitarisée	19
4.5	Service Suricata actif sur pfSense – Surveillance des interfaces LAN et DMZ	19
4.6	Tableau de bord principal Wazuh – Vue globale des événements de sécurité	20
4.7	Agents Wazuh actifs – Connexion des endpoints au Wazuh Manager	20
4.8	Détails des agents Wazuh déployés sur les machines de la DMZ	21
4.9	Vue synthétique Wazuh – Distribution des événements par niveau de criticité	21
4.10	Vue synthétique Wazuh – Évolution temporelle des alertes de sécurité	21
4.11	Interface DVWA – Application web vulnérable déployée dans la DMZ pour les tests d'intrusion	22
4.12	Workflow SOAR Shuffle – Automatisation de la chaîne Wazuh → Shuffle → TheHive	22
5.1	Scan Nmap depuis Kali Linux – Reconnaissance des ports ouverts sur le serveur DVWA (DMZ)	23
5.2	Attaque brute-force SSH depuis Kali Linux – Tentative d'authentification par dictionnaire	24
5.3	Alertes Suricata sur pfSense – Détection des activités malveillantes sur le réseau	25
5.4	Événement brute-force SSH détecté par Wazuh – Alerte de niveau critique générée	25

5.5	Événements Suricata intégrés dans Wazuh – Corrélation des alertes réseau et hôte	26
5.6	Tableau de bord Wazuh – Événements de sécurité sur la machine Windows (DMZ)	26
5.7	Tableau de bord Wazuh – Événements de sécurité sur la zone DMZ . .	27
5.8	Alertes reçues dans TheHive – Transmises automatiquement par le workflow Shuffle	27
5.9	Cas d’incident dans TheHive – Détails, observables et statut de traitement	28
5.10	Cas marqué dans TheHive – Confirmation d’un vrai positif et prise en charge par l’analyste	28

Liste des tableaux

2.1	Composantes technologiques d'un SOC et leur rôle	11
2.2	Comparaison solutions open source vs propriétaires	13
3.1	Plan d'adressage IP de l'infrastructure SOC	16
3.2	Environnement matériel et logiciel du projet	16
5.1	Synthèse des résultats de détection et de réponse par scénario d'attaque	28
5.2	Analyse qualitative des performances des composants du SOC	29

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

Dans un paysage numérique marqué par une croissance exponentielle des cybermenaces, les organisations sont confrontées à des attaques de plus en plus sophistiquées ciblant leurs systèmes d'information. Les ransomwares, les intrusions APT (Advanced Persistent Threats), les attaques par force brute ou encore les campagnes de phishing font désormais partie du quotidien des équipes sécurité. Face à cette réalité, la mise en place d'un Centre Opérationnel de Sécurité (SOC) est devenue une nécessité stratégique pour toute organisation souhaitant assurer la continuité et la résilience de ses activités.

Un SOC est une unité centralisée regroupant des personnes, des processus et des technologies dédiés à la surveillance continue de l'infrastructure informatique d'une organisation, à la détection des incidents de sécurité, à leur analyse et à la coordination de la réponse. La figure 1.1 illustre le cycle fondamental d'un SOC.

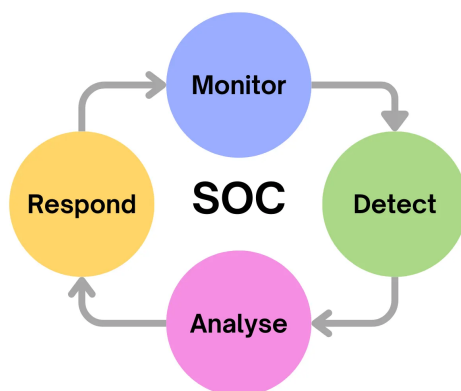


FIGURE 1.1 – Cycle opérationnel d'un SOC : Monitor – Detect – Analyse – Respond

Ce projet s'inscrit dans ce contexte en proposant un déploiement complet d'un SOC open source, permettant ainsi à une organisation de disposer de capacités de surveillance et de réponse aux incidents sans dépendre de solutions propriétaires coûteuses.

1.2 Problématique

La question centrale de ce projet est la suivante : *comment concevoir, déployer et valider un SOC fonctionnel basé exclusivement sur des outils open source, capable de couvrir l'intégralité du cycle Monitor–Detect–Analyse–Respond dans un environnement réseau segmenté ?*

Cette problématique soulève plusieurs défis techniques : l'intégration cohérente d'outils hétérogènes, la segmentation réseau (LAN/DMZ), la corrélation des alertes entre le pare-feu et le SIEM, ainsi que l'automatisation de la réponse aux incidents.

1.3 Objectifs du Projet

Les objectifs principaux de ce projet sont :

- Concevoir une architecture réseau segmentée (WAN, LAN, DMZ) et la déployer sous pfSense ;
- Intégrer Suricata comme moteur de détection d'intrusions au niveau du pare-feu ;
- Déployer Wazuh pour la collecte centralisée des journaux et la corrélation des événements de sécurité ;
- Mettre en place TheHive pour la gestion structurée des cas d'incidents ;
- Automatiser les workflows de réponse aux incidents avec Shuffle (SOAR) ;
- Valider l'ensemble de la chaîne par des simulations d'attaques réelles.

1.4 Structure du Rapport

Ce rapport est organisé en cinq chapitres. Le chapitre 2 présente l'état de l'art et les concepts théoriques associés au SOC et aux outils utilisés. Le chapitre 3 décrit l'architecture globale de la solution. Le chapitre 4 détaille la configuration et le déploiement de chaque composant. Le chapitre 5 présente les tests, les résultats et la validation de la solution. Enfin, une conclusion générale clôture le rapport.

Chapitre 2

État de l'Art et Cadre Théorique

2.1 Le Security Operations Center (SOC)

2.1.1 Définition et Rôle

Un Centre Opérationnel de Sécurité (SOC) est une infrastructure organisationnelle et technique permettant la surveillance en temps réel des systèmes d'information, la détection des incidents de sécurité, l'analyse forensique et la coordination de la réponse. Il constitue le nœud central de la cybersécurité opérationnelle d'une organisation.

2.1.2 Composantes Technologiques d'un SOC

Les principales technologies d'un SOC moderne sont regroupées dans le tableau [2.1](#).

TABLE 2.1 – Composantes technologiques d'un SOC et leur rôle

Technologie	Catégorie	Rôle dans le SOC
SIEM	Security Information and Event Management	Collecte, normalisation, corrélation et archivage des journaux d'événements
IDS/IPS	Intrusion Detection / Prevention System	Détection et blocage d'activités malveillantes sur le réseau ou les hôtes
EDR	Endpoint Detection and Response	Surveillance et réponse sur les terminaux
SOAR	Security Orchestration, Automation and Response	Automatisation des processus de réponse aux incidents
Pare-feu	Firewall / Next-Generation Firewall	Filtrage et contrôle du trafic réseau
Gestion des cas	Case Management	Suivi, documentation et gestion du cycle de vie des incidents

2.2 Présentation des Outils Utilisés

2.2.1 pfSense

pfSense est un pare-feu et routeur open source basé sur FreeBSD. Il offre des fonctionnalités avancées de filtrage de paquets (via PF), de VPN, de NAT, ainsi que la possibilité d'intégrer des modules supplémentaires tels que Suricata pour la détection d'intrusions.

2.2.2 Suricata

Suricata est un moteur open source de détection d'intrusions réseau (NIDS), de prévention d'intrusions (IPS) et de surveillance du trafic réseau (NSM). Il utilise un ensemble de règles (Emerging Threats, Snort, etc.) pour analyser le trafic en temps réel et générer des alertes.

2.2.3 Wazuh

Wazuh est une plateforme open source SIEM/EDR offrant des capacités de collecte de journaux (agents déployés sur les hôtes), d'analyse et de corrélation d'événements, de détection de vulnérabilités, de surveillance de l'intégrité des fichiers (FIM) et d'intégration avec Elasticsearch pour le stockage et la visualisation.

2.2.4 TheHive

TheHive est une plateforme open source de gestion des incidents de sécurité (Case Management). Elle permet de créer, documenter et suivre les cas d'incidents, d'y associer des observables (IOC) et de collaborer entre analystes SOC.

2.2.5 Shuffle

Shuffle est une plateforme SOAR (Security Orchestration, Automation and Response) open source permettant de créer des workflows automatisés pour la réponse aux incidents. Elle s'intègre avec Wazuh, TheHive et de nombreux autres outils via des connecteurs API.

2.2.6 DVWA

Damn Vulnerable Web Application (DVWA) est une application web volontairement vulnérable déployée dans la DMZ à des fins de tests. Elle permet de simuler des attaques web réelles dans un environnement contrôlé.

2.2.7 Kali Linux

Kali Linux est une distribution Linux dédiée aux tests de pénétration et à la sécurité offensive. Elle est utilisée dans ce projet comme machine attaquante pour simuler des scénarios d'intrusion.

2.3 Comparaison avec des Solutions Propriétaires

Le tableau [2.2](#) compare les solutions open source retenues avec leurs équivalents propriétaires.

TABLE 2.2 – Comparaison solutions open source vs propriétaires

Catégorie	Open Source (retenu)	Équivalent Propriétaire	Avantage Open Source
Pare-feu	pfSense	Cisco ASA, Fortinet	Coût zéro, communauté active
NIDS	Suricata	Snort (Cisco), Darktrace	Performances multi- thread, règles ET
SIEM / EDR	Wazuh	Splunk, IBM QRadar	Agents gratuits, intégration Elasticsearch
Case Management	TheHive	ServiceNow, IBM Resilient	API riche, intégration MISP
SOAR	Shuffle	Palo Alto XSOAR	Workflows visuels, no- code

Chapitre 3

Architecture de la Solution

3.1 Vue Globale de l'Architecture

L'architecture déployée est une infrastructure réseau virtualisée sous VMware, segmentée en trois zones : WAN (accès Internet), LAN (réseau interne sécurisé) et DMZ (zone démilitarisée exposant des services). La figure 3.1 présente l'architecture réseau globale.

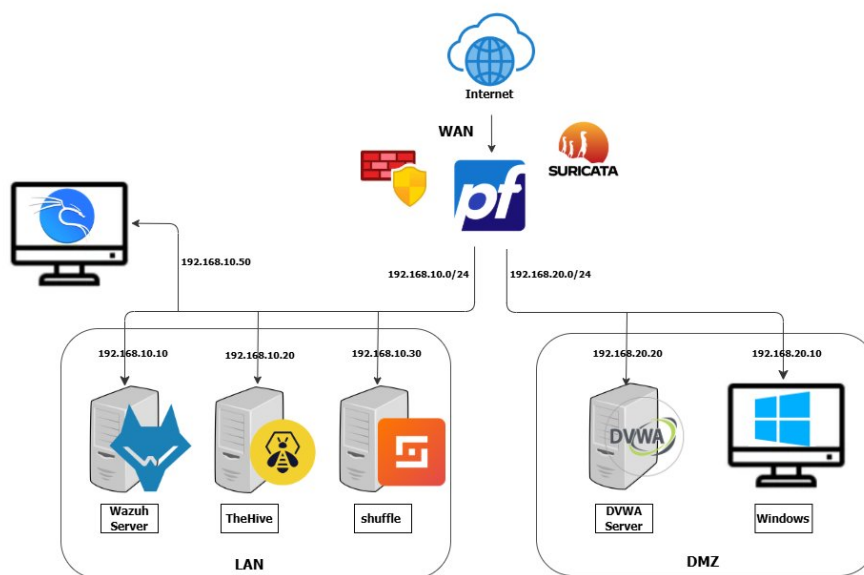


FIGURE 3.1 – Architecture réseau globale du SOC (LAN, DMZ, WAN)

pfSense joue le rôle de passerelle centrale entre les trois zones. Le LAN héberge les outils de surveillance (Wazuh, TheHive, Shuffle) ainsi que la machine attaquante Kali Linux. La DMZ héberge les cibles d'attaque (serveur DVWA et machine Windows).

3.2 Flux de Données du SOC

La figure 3.2 illustre le flux de données entre les différents composants du SOC.

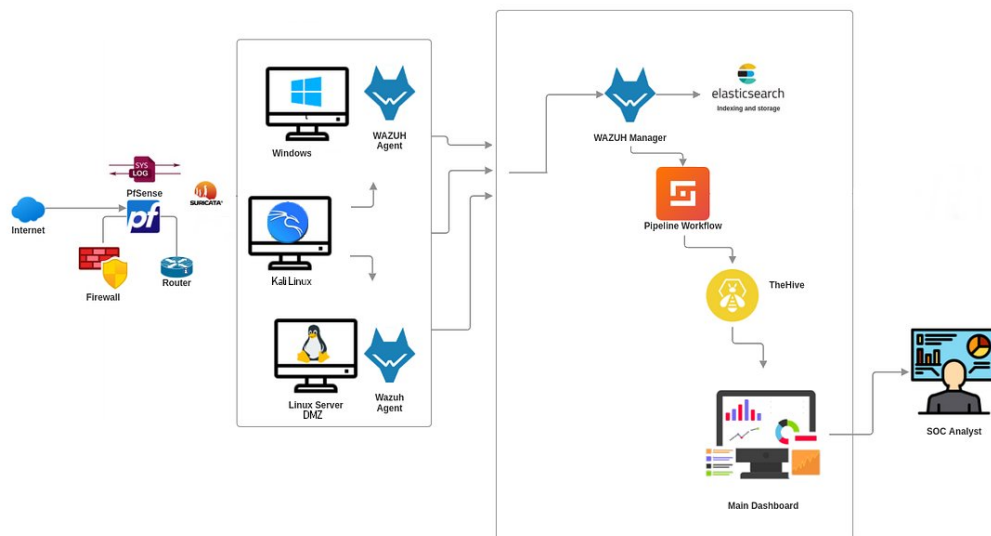


FIGURE 3.2 – Flux de données entre les composants du SOC (Wazuh, Shuffle, TheHive, Elasticsearch)

Les agents Wazuh déployés sur les machines Windows et Linux (DMZ) collectent les journaux et les transmettent au Wazuh Manager. Ce dernier corrèle les événements, stocke les données dans Elasticsearch et déclenche des alertes vers Shuffle. Shuffle orchestre la réponse en créant automatiquement des cas dans TheHive, permettant à l'analyste SOC de les traiter depuis le tableau de bord centralisé.

3.3 Adressage IP et Segmentation Réseau

Le tableau 3.1 récapitule le plan d'adressage IP de l'infrastructure.

TABLE 3.1 – Plan d’adressage IP de l’infrastructure SOC

Machine / Service	Zone	Adresse IP	Rôle
pfSense WAN	WAN	192.168.220.135/24	Passerelle Internet (DHCP)
pfSense LAN	LAN	192.168.10.1/24	Passerelle LAN
pfSense DMZ	DMZ	192.168.20.1/24	Passerelle DMZ
Wazuh Server	LAN	192.168.10.10	SIEM / EDR Manager
TheHive	LAN	192.168.10.20	Gestion des incidents
Shuffle	LAN	192.168.10.30	Orchestration SOAR
Kali Linux	LAN	192.168.10.50	Machine attaquante
DVWA Server	DMZ	192.168.20.20	Cible web vulnérable
Windows	DMZ	192.168.20.10	Cible endpoint

3.4 Matériel et Logiciels

Le tableau 3.2 résume l’environnement de virtualisation utilisé.

TABLE 3.2 – Environnement matériel et logiciel du projet

Composant	Détails
Hyperviseur	VMware Workstation Pro
Système pfSense	pfSense 2.8.1-RELEASE (amd64), FreeBSD
SIEM	Wazuh 4.x
Indexeur	Elasticsearch 8.x
Case Management	TheHive 5.x
SOAR	Shuffle (dernière version stable)
IDS/IPS	Suricata (paquet pfSense)
Machine attaquante	Kali Linux (dernière version)
Cible web	DVWA (Damn Vulnerable Web Application)

Chapitre 4

Déploiement et Configuration

4.1 Configuration de pfSense

4.1.1 Interfaces Réseau

pfSense est configuré avec trois interfaces réseau comme indiqué sur la console FreeBSD (figure 4.1) :

- em0 : Interface WAN, adresse IP 192.168.220.135/24 obtenue en DHCP ;
- em1 : Interface LAN, adresse IP statique 192.168.10.1/24 ;
- em2 : Interface DMZ (opt1), adresse IP statique 192.168.20.1/24.

```
Enter an option:

FreeBSD/amd64 (pfSense.home.arpa) (ttyv0)

VMware Virtual Machine - Netgate Device ID: 2647958fc9468d806040

*** Welcome to pfSense 2.8.1-RELEASE (amd64) on pfSense ***

WAN (wan) -> em0 -> v4/DHCP4: 192.168.220.135/24
LAN (lan) -> em1 -> v4: 192.168.10.1/24
DMZ (opt1) -> em2 -> v4: 192.168.20.1/24

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                10) Filter Logs
2) Set interface(s) IP address      11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                    14) Disable Secure Shell (sshd)
6) Halt system                      15) Restore recent configuration
7) Ping host                        16) Restart PHP-FPM
8) Shell

Enter an option: █
```

FIGURE 4.1 – Console pfSense 2.8.1 – Configuration des interfaces WAN, LAN et DMZ

4.1.2 Règles de Pare-feu

La politique de filtrage applique le principe du moindre privilège : tout trafic non explicitement autorisé est bloqué. Les figures 4.2 à 4.4 illustrent les règles configurées

sur chaque interface.

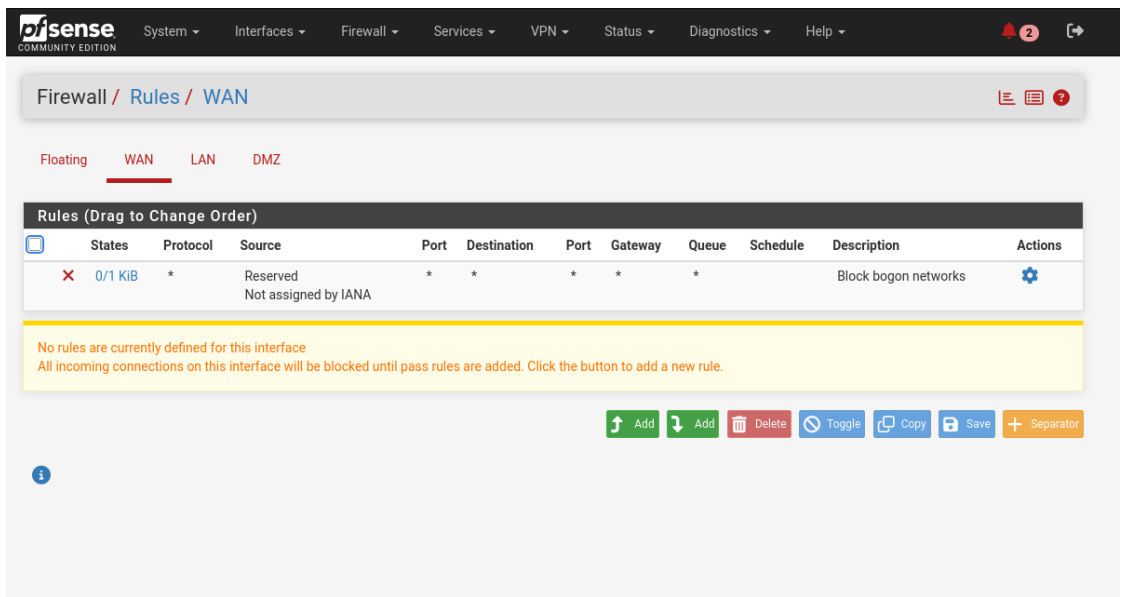


FIGURE 4.2 – Règles de pare-feu sur l’interface WAN – Filtrage du trafic entrant depuis Internet

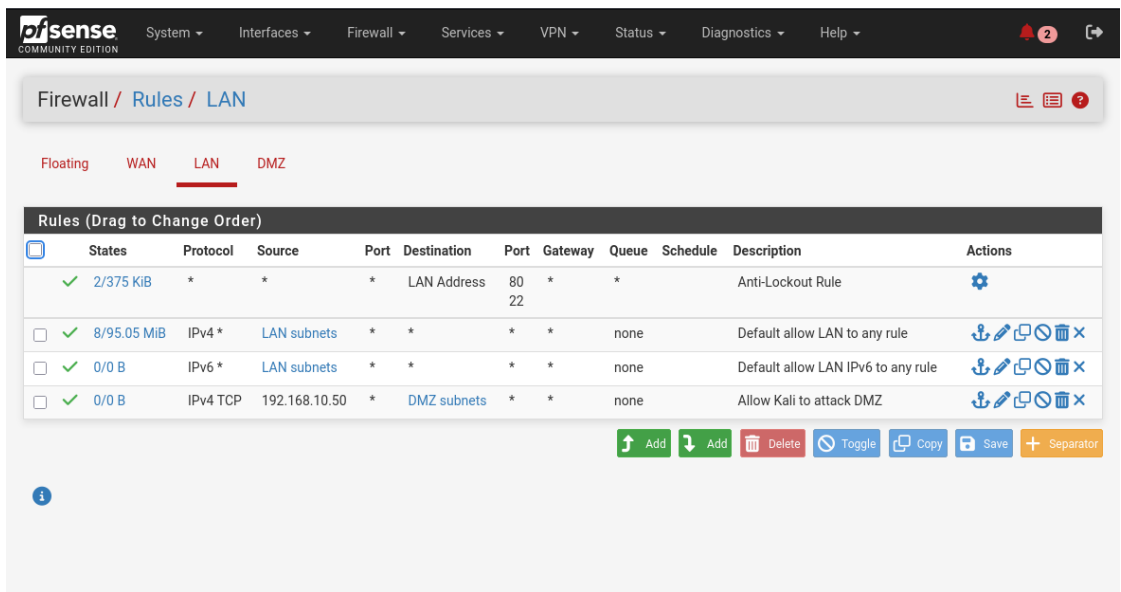


FIGURE 4.3 – Règles de pare-feu sur l’interface LAN – Contrôle des accès depuis le réseau interne

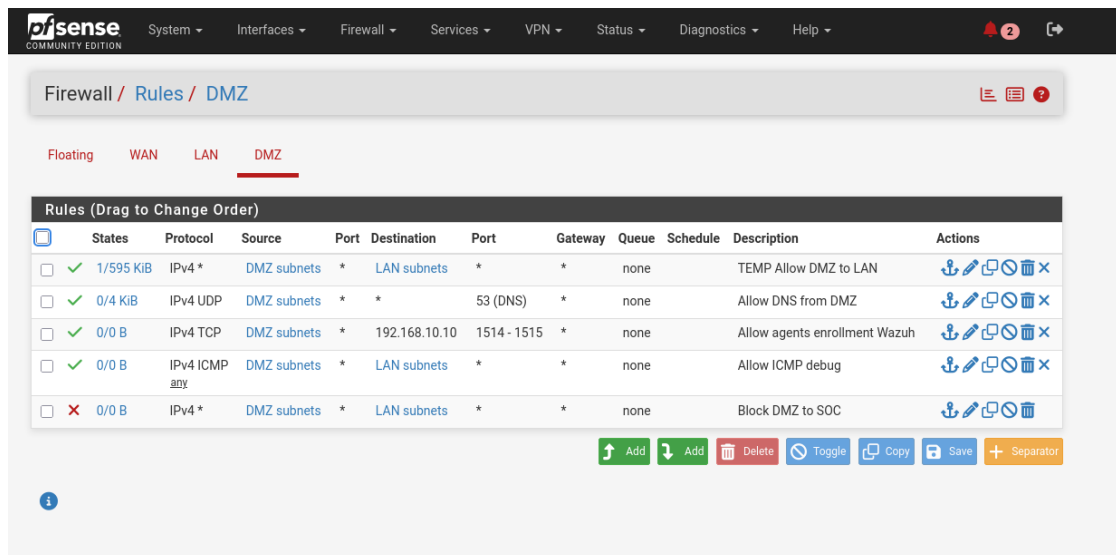


FIGURE 4.4 – Règles de pare-feu sur l’interface DMZ – Isolation de la zone démilitarisée

4.1.3 Configuration de Suricata sur pfSense

Suricata est installé comme paquet complémentaire sur pfSense. Il est configuré en mode IDS/IPS sur les interfaces LAN et DMZ. La figure 4.5 montre l’état des services Suricata actifs.

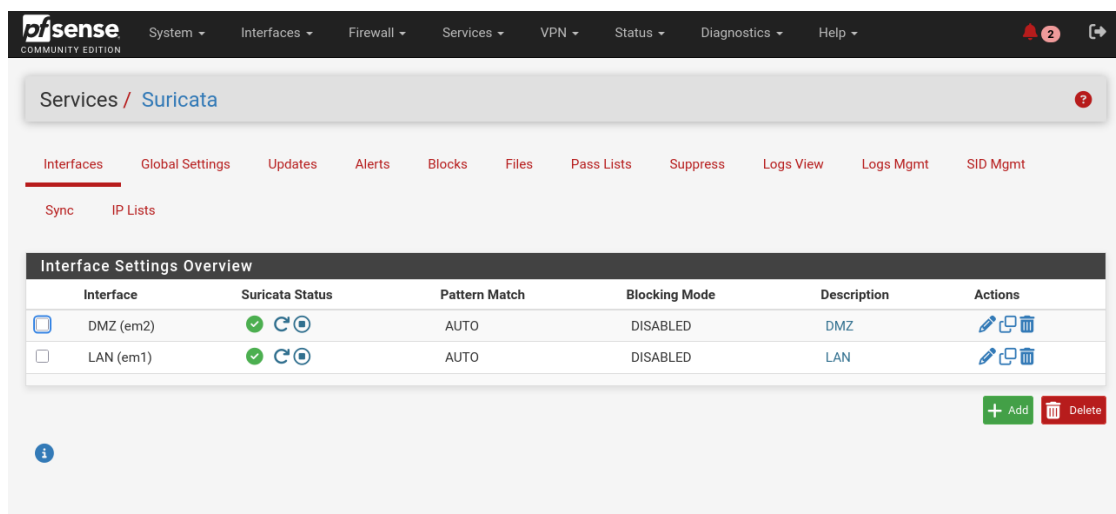


FIGURE 4.5 – Service Suricata actif sur pfSense – Surveillance des interfaces LAN et DMZ

4.2 Déploiement de Wazuh

4.2.1 Architecture Wazuh

Wazuh est déployé selon une architecture Manager/Agent. Le Wazuh Manager (192.168.10.10) centralise les alertes de tous les agents. Des agents Wazuh sont installés

sur :

- La machine Windows (DMZ – 192.168.20.10) ;
- Le serveur Linux DMZ (192.168.20.20 – hébergeant DVWA).

4.2.2 Tableau de Bord Wazuh

La figure 4.6 présente le tableau de bord principal de Wazuh affichant une vue globale des événements de sécurité.

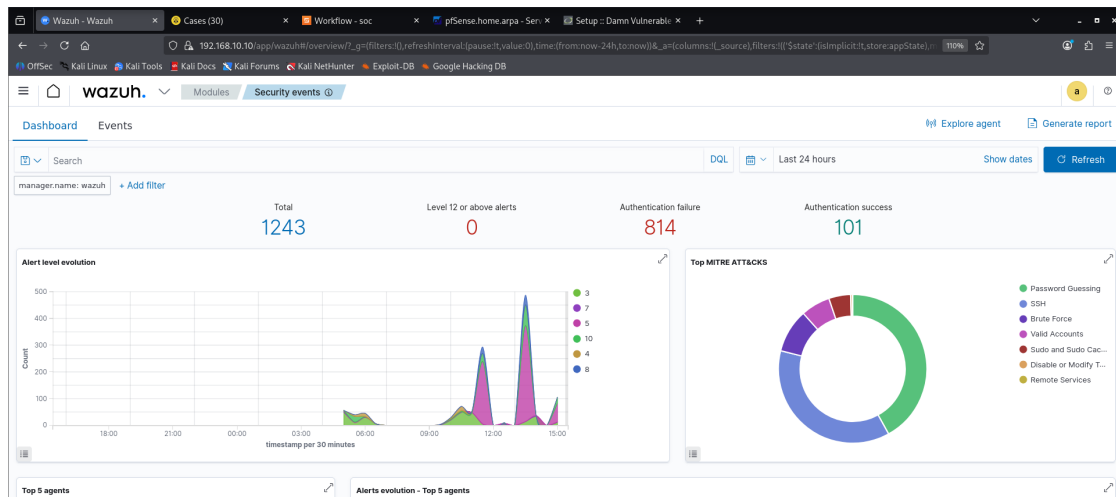


FIGURE 4.6 – Tableau de bord principal Wazuh – Vue globale des événements de sécurité

4.2.3 Agents Actifs

La figure 4.7 montre la liste des agents Wazuh actifs et leur état de connexion au Manager.

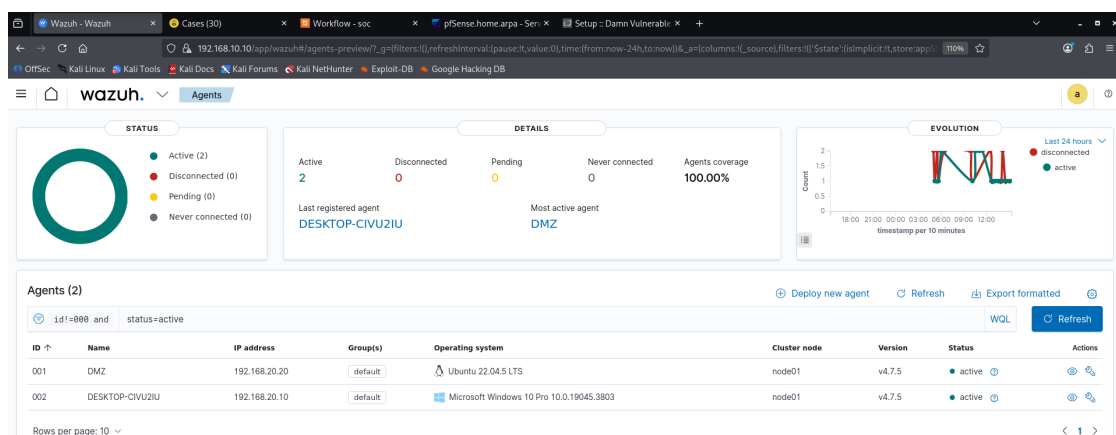
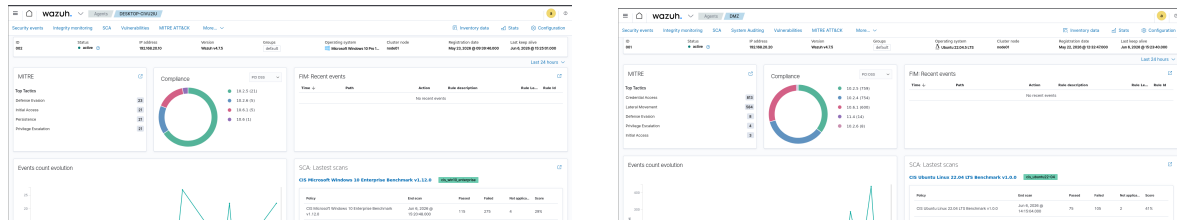


FIGURE 4.7 – Agents Wazuh actifs – Connexion des endpoints au Wazuh Manager

4.2.4 Agent Windows et DMZ

Les figures 4.8a et 4.8b illustrent respectivement les détails de l'agent Windows et de l'agent DMZ Linux.



(a) Agent Windows (192.168.20.10)

(b) Agent Linux DMZ (192.168.20.20)

FIGURE 4.8 – Détails des agents Wazuh déployés sur les machines de la DMZ

4.2.5 Vues Globales du Tableau de Bord

Les figures 4.9 et 4.10 présentent des vues synthétiques du tableau de bord Wazuh avec les statistiques d'événements et les niveaux d'alerte.

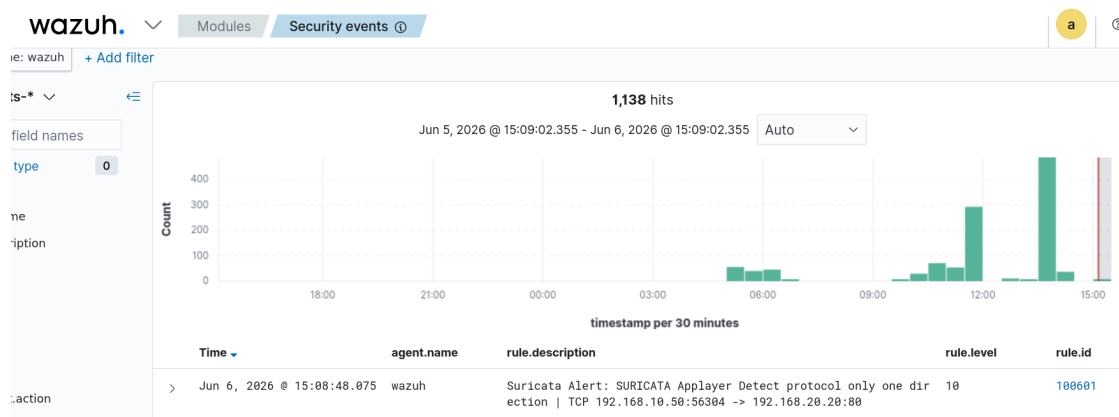


FIGURE 4.9 – Vue synthétique Wazuh – Distribution des événements par niveau de criticité

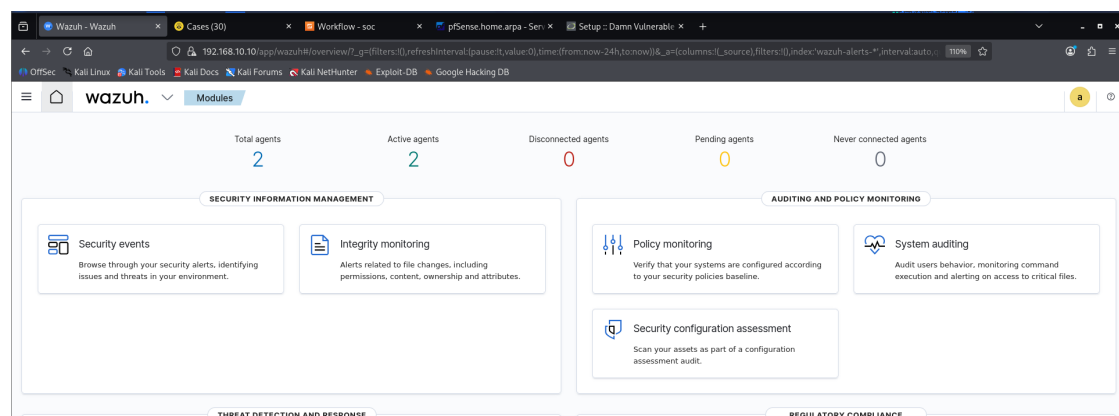


FIGURE 4.10 – Vue synthétique Wazuh – Évolution temporelle des alertes de sécurité

4.3 Déploiement de DVWA

DVWA (Damn Vulnerable Web Application) est installé sur le serveur Linux de la DMZ. La figure 4.11 montre l'interface de connexion de DVWA, accessible depuis le LAN pour les tests d'attaque.

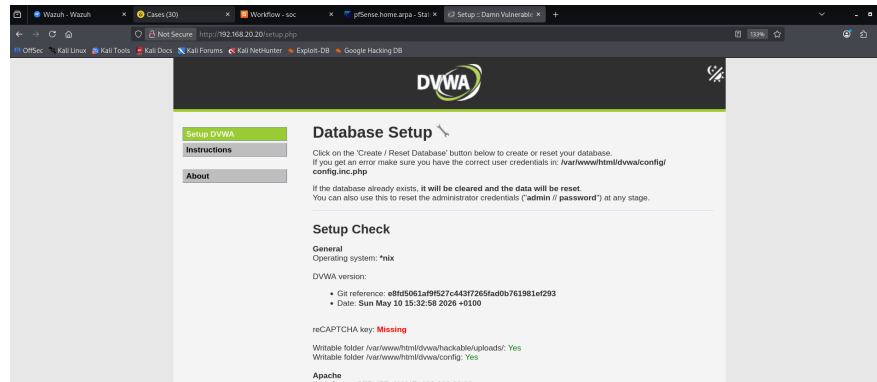


FIGURE 4.11 – Interface DVWA – Application web vulnérable déployée dans la DMZ pour les tests d'intrusion

4.4 Configuration du Workflow Shuffle

Shuffle est configuré pour recevoir les alertes de Wazuh via webhook et créer automatiquement des cas dans TheHive. La figure 4.12 présente le workflow SOAR implémenté.

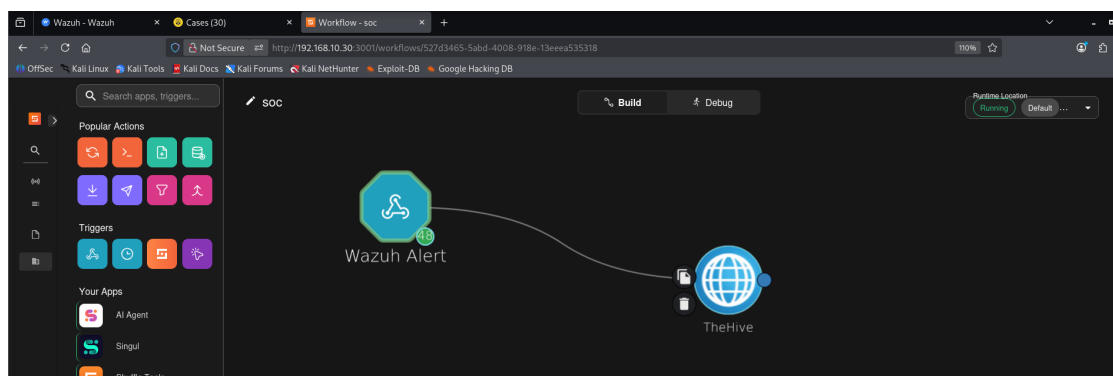


FIGURE 4.12 – Workflow SOAR Shuffle – Automatisation de la chaîne Wazuh → Shuffle → TheHive

Le workflow réalise les actions suivantes :

1. Réception d'une alerte Wazuh via webhook HTTP ;
2. Extraction des observables (IP source, règle déclenchée, niveau d'alerte) ;
3. Enrichissement optionnel via des sources de Threat Intelligence ;
4. Création automatique d'un cas dans TheHive avec les informations de l'alerte.

Chapitre 5

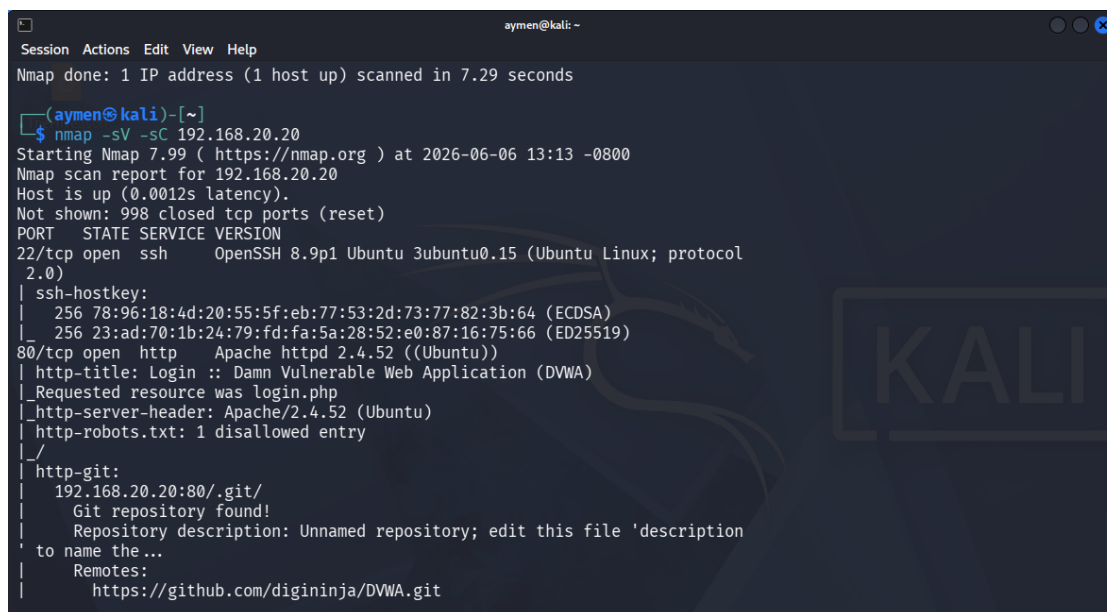
Tests, Résultats et Validation

5.1 Scénarios d'Attaque Simulés

Pour valider la chaîne de détection et de réponse du SOC, deux scénarios d'attaque ont été simulés depuis la machine Kali Linux (192.168.10.50) vers les cibles de la DMZ.

5.1.1 Scénario 1 : Scan Nmap (Reconnaissance)

Un scan Nmap a été lancé depuis Kali Linux vers le serveur DVWA (192.168.20.20) afin de détecter les ports ouverts et les services exposés. La figure 5.1 montre la commande exécutée et les résultats obtenus.



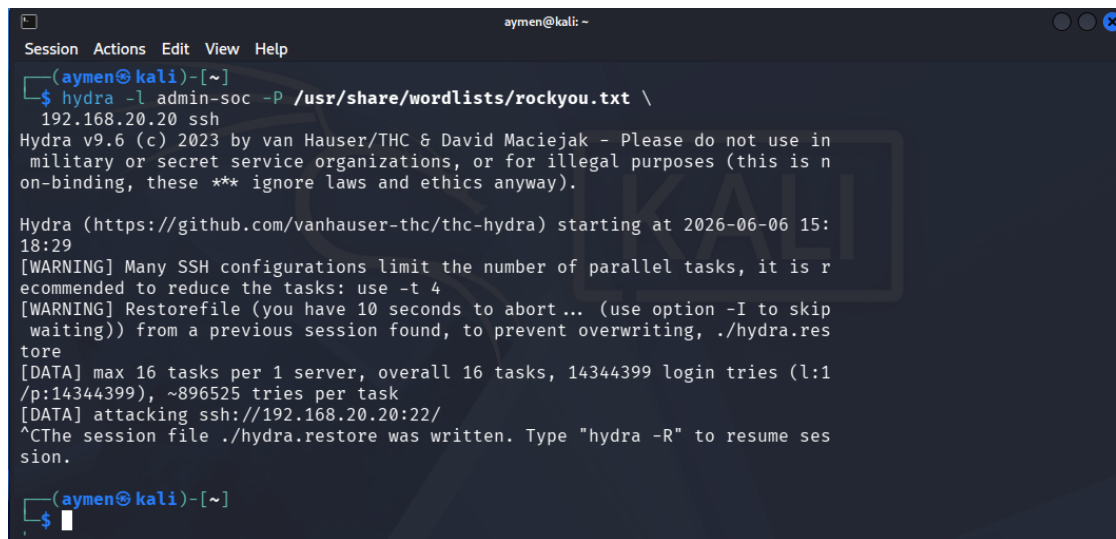
```
aymen@kali: ~  
Session Actions Edit View Help  
Nmap done: 1 IP address (1 host up) scanned in 7.29 seconds  
  
(aymen@kali)-[~]  
$ nmap -sV -sC 192.168.20.20  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-06 13:13 -0800  
Nmap scan report for 192.168.20.20  
Host is up (0.0012s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 78:96:18:4d:20:55:5f:eb:77:53:2d:73:77:82:3b:64 (ECDSA)  
|   256 23:ad:70:1b:24:79:fd:fa:5a:28:52:e0:87:16:75:66 (ED25519)  
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))  
|_ http-title: Login :: Damn Vulnerable Web Application (DVWA)  
|_ Requested resource was login.php  
|_ http-server-header: Apache/2.4.52 (Ubuntu)  
|_ http-robots.txt: 1 disallowed entry  
|_  
|_ http-git:  
|   192.168.20.20:80/.git/  
|   Git repository found!  
|   Repository description: Unnamed repository; edit this file 'description'  
|   to name the...  
|   Remotes:  
|     https://github.com/digininja/DVWA.git
```

FIGURE 5.1 – Scan Nmap depuis Kali Linux – Reconnaissance des ports ouverts sur le serveur DVWA (DMZ)

5.1.2 Scénario 2 : Brute-Force SSH

Une attaque par force brute SSH a été lancée depuis Kali Linux vers la machine Windows de la DMZ, en utilisant un dictionnaire de mots de passe. La figure 5.2 illustre

l'attaque en cours.



```

aymen@kali: ~
Session Actions Edit View Help
(aymen@kali)-[~]
$ hydra -l admin-soc -P /usr/share/wordlists/rockyou.txt \
192.168.20.20 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-06 15:
18:29
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
waiting)) from a previous session found, to prevent overwriting, ./hydra.res
tore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1
/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.20.20:22/
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume ses
sion.

(aymen@kali)-[~]
$

```

FIGURE 5.2 – Attaque brute-force SSH depuis Kali Linux – Tentative d'authentification par dictionnaire

5.2 Résultats de Détection

5.2.1 Alertes Suricata sur pfSense

Suricata a détecté et généré des alertes pour les deux scénarios d'attaque. La figure 5.3 présente les alertes Suricata enregistrées dans pfSense.

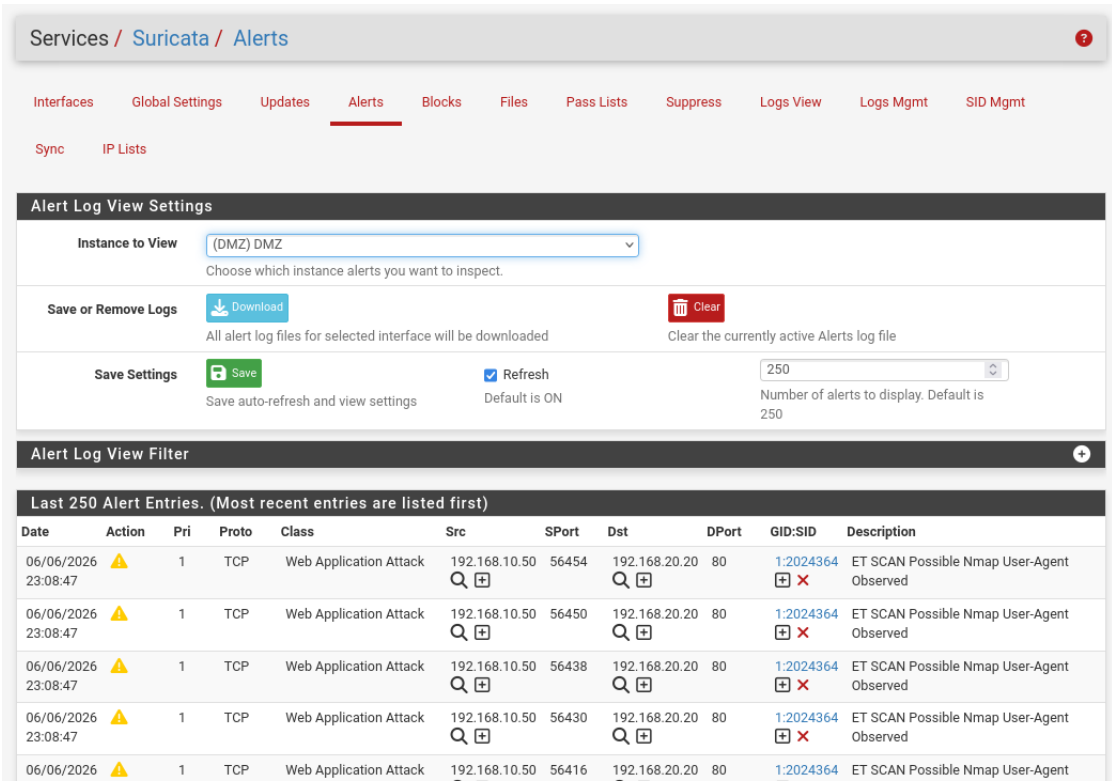


FIGURE 5.3 – Alertes Suricata sur pfSense – Détection des activités malveillantes sur le réseau

5.2.2 Événements Wazuh – Brute-Force Détecté

L'agent Wazuh déployé sur la machine Windows a détecté les tentatives répétées de connexion SSH échouées et généré des alertes de niveau critique. La figure 5.4 montre l'événement de brute-force capturé par Wazuh.

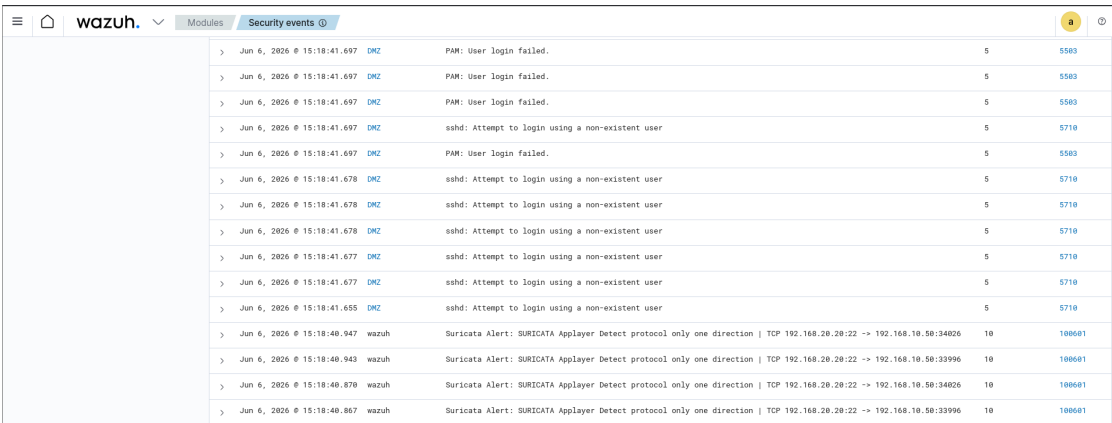


FIGURE 5.4 – Événement brute-force SSH détecté par Wazuh – Alerte de niveau critique générée

5.2.3 Événements Suricata dans Wazuh

Wazuh intègre également les alertes générées par Suricata (via syslog), permettant une corrélation croisée des événements réseau et hôte. La figure 5.5 montre ces événements corrélés.

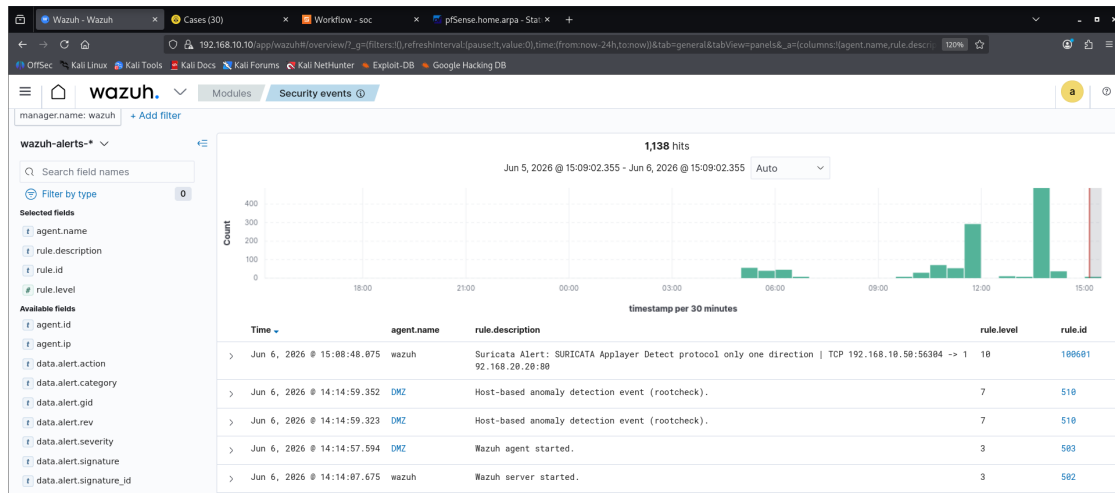


FIGURE 5.5 – Événements Suricata intégrés dans Wazuh – Corrélation des alertes réseau et hôte

5.2.4 Tableaux de Bord de Sécurité par Zone

Les figures 5.6 et 5.7 présentent les tableaux de bord dédiés aux événements de sécurité respectivement sur la machine Windows et sur la zone DMZ.

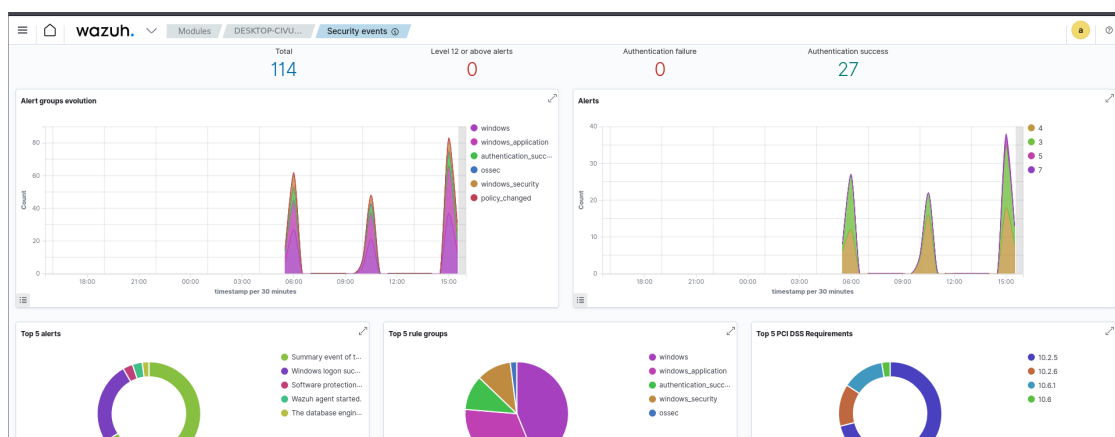


FIGURE 5.6 – Tableau de bord Wazuh – Événements de sécurité sur la machine Windows (DMZ)

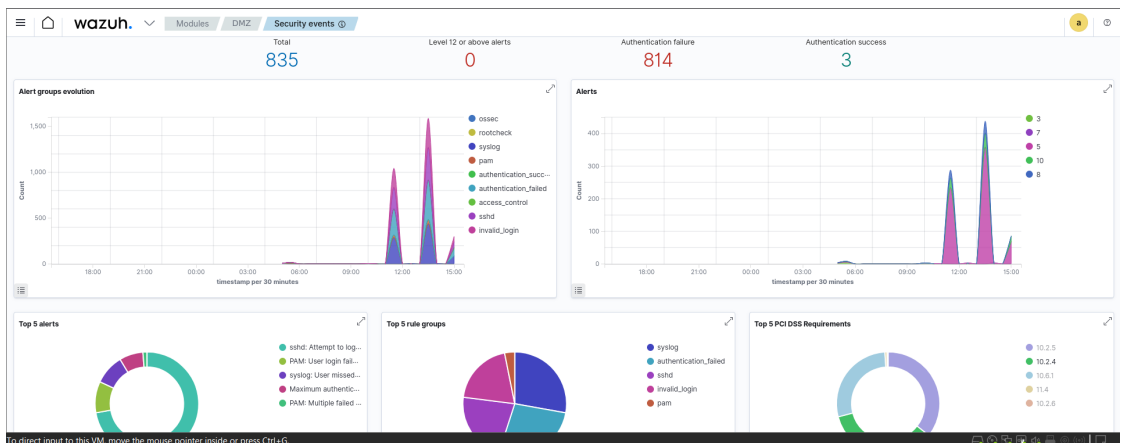


FIGURE 5.7 – Tableau de bord Wazuh – Événements de sécurité sur la zone DMZ

5.3 Gestion des Incidents avec TheHive

5.3.1 Alertes Reçues dans TheHive

Suite à la détection par Wazuh et l'orchestration par Shuffle, des alertes ont été automatiquement transmises à TheHive. La figure 5.8 montre la liste des alertes reçues.

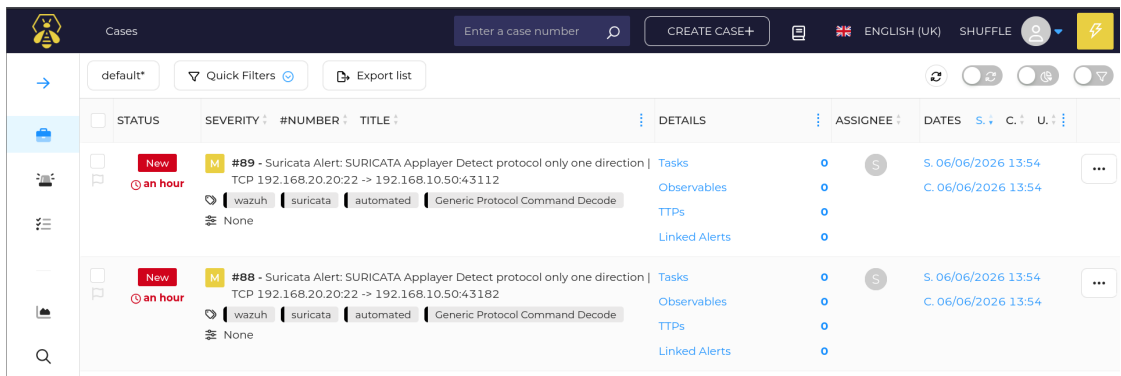


FIGURE 5.8 – Alertes reçues dans TheHive – Transmises automatiquement par le workflow Shuffle

5.3.2 Cas d'Incident Créé

La figure 5.9 présente un cas d'incident créé dans TheHive avec les détails de l'alerte, les observables associés et le statut de traitement.

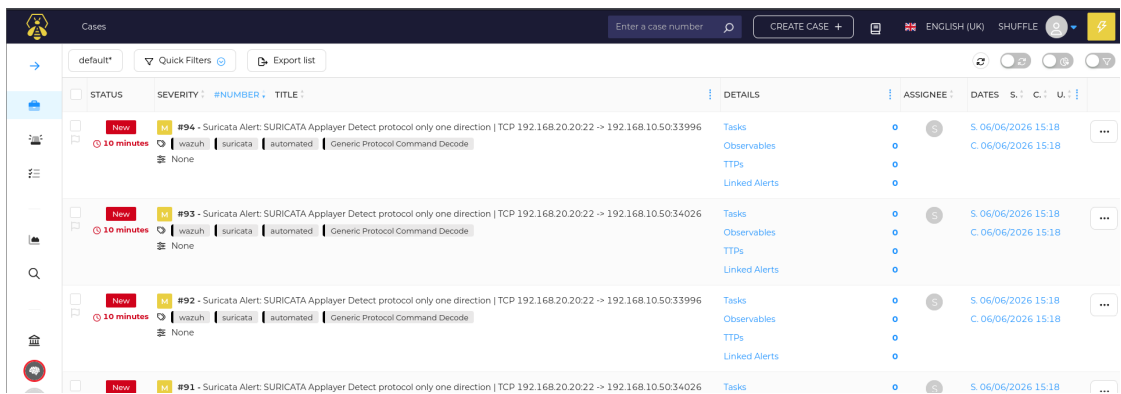


FIGURE 5.9 – Cas d’incident dans TheHive – Détails, observables et statut de traitement

5.3.3 Marquage et Clôture du Cas

La figure 5.10 illustre un cas marqué (flagged) dans TheHive, indiquant qu’il a été identifié comme un vrai positif et pris en charge par l’analyste.

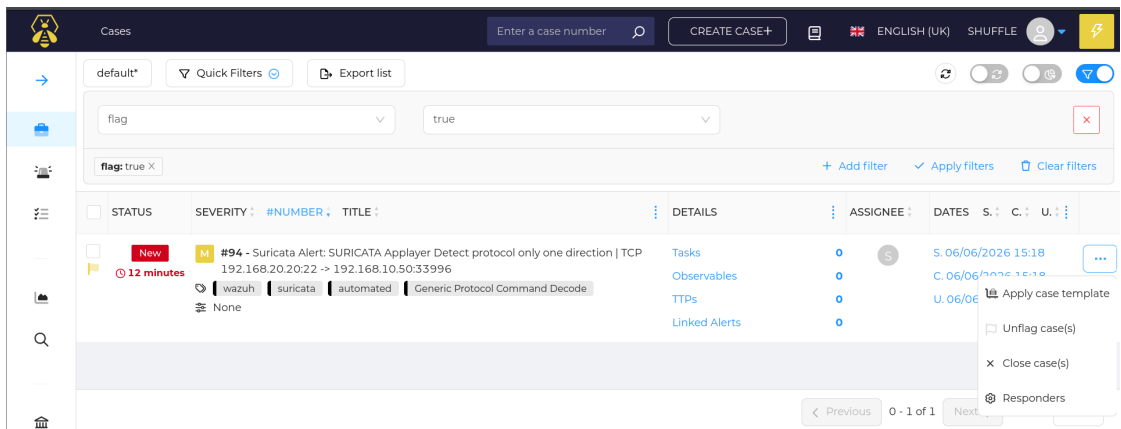


FIGURE 5.10 – Cas marqué dans TheHive – Confirmation d’un vrai positif et prise en charge par l’analyste

5.4 Synthèse des Résultats

Le tableau 5.1 récapitule les résultats de validation pour chacun des scénarios d’attaque testés.

TABLE 5.1 – Synthèse des résultats de détection et de réponse par scénario d’attaque

Scénario	Suricata	Wazuh	Shuffle	TheHive	Résultat
Scan Nmap	✓	✓	✓	✓	Détecté
Brute-Force SSH	✓	✓	✓	✓	Détecté

Les deux scénarios d'attaque ont été détectés avec succès par l'ensemble de la chaîne SOC. Les alertes ont été propagées de Suricata/Wazuh jusqu'à TheHive en passant par Shuffle, validant ainsi l'intégration complète du cycle Monitor–Detect–Analyse–Respond.

5.5 Analyse des Performances

Le tableau 5.2 présente une analyse qualitative des performances de chaque composant.

TABLE 5.2 – Analyse qualitative des performances des composants du SOC

Composant	Points Forts	Limitations Observées	Pistes d'Amélioration
pfSense / Suricata	Détection réseau temps réel, règles ET	Charge CPU élevée en mode IPS	Activer le mode multi-thread
Wazuh	Corrélation multi-source, FIM intégré	Latence sur grands volumes	Tuning des règles de corrélation
Shuffle	Automatisation rapide, no-code	Dépendance API externe	Ajouter Threat Intelligence
TheHive	Gestion structurée des cas	Interface parfois lente	Mise à jour vers TheHive 5.x

Conclusion Générale et Perspectives

Ce projet a permis de concevoir et de déployer un SOC fonctionnel entièrement basé sur des outils open source, dans un environnement réseau segmenté et virtualisé. L'architecture implémentée couvre intégralement le cycle opérationnel d'un SOC moderne : surveillance continue (Monitor), détection des menaces (Detect), analyse des incidents (Analyse) et réponse automatisée (Respond).

Les principaux résultats atteints sont :

- La mise en place d'une segmentation réseau WAN/LAN/DMZ sur pfSense avec des règles de filtrage strictes ;
- L'intégration de Suricata comme IDS/IPS au niveau du pare-feu pour la détection réseau en temps réel ;
- Le déploiement de Wazuh comme SIEM/EDR centralisant les journaux de tous les endpoints ;
- La création de workflows SOAR automatisés avec Shuffle reliant Wazuh et TheHive ;
- La validation de la chaîne complète par des simulations d'attaques réelles (scan Nmap, brute-force SSH).

Perspectives d'amélioration :

Pour renforcer davantage ce SOC, plusieurs pistes peuvent être envisagées :

- Intégration d'une plateforme de Threat Intelligence (MISP) pour enrichir les alertes avec des IOC ;
- Mise en place de règles de détection comportementale avancée (ML-based) dans Wazuh ;
- Ajout d'un honeypot pour la détection précoce des attaques de type reconnaissance ;
- Extension de la couverture de monitoring aux applications web (WAF) ;
- Implémentation d'une réponse active automatique (blocage IP via pfSense API depuis Shuffle).

Ce projet constitue une base solide pour toute organisation souhaitant déployer un SOC opérationnel avec un budget limité, tout en maintenant un niveau de maturité sécurité élevé.

Références Bibliographiques

- [1] Documentation officielle pfSense, *pfSense Documentation*, Netgate, 2024. <https://docs.netgate.com/pfsense/en/latest/>
- [2] OISF, *Suricata User Guide*, Open Information Security Foundation, 2024. <https://suricata.readthedocs.io/>
- [3] Wazuh Inc., *Wazuh Documentation*, 2024. <https://documentation.wazuh.com/>
- [4] StrangeBee, *TheHive Documentation*, 2024. <https://docs.strangebee.com/thehive/>
- [5] Shuffle, *Shuffle Documentation – Open Source SOAR*, 2024. <https://shuffler.io/docs>
- [6] SANS Institute, *Building a World-Class Security Operations Center*, SANS Reading Room, 2023.
- [7] MITRE, *ATT&CK Framework for Enterprise*, MITRE Corporation, 2024. <https://attack.mitre.org/>
- [8] Bejtlich, R., *The Practice of Network Security Monitoring*, No Starch Press, 2013.
- [9] Cichonski, P. et al., *Computer Security Incident Handling Guide*, NIST SP 800-61 Rev. 2, 2012.

Annexe A

Récapitulatif des Commandes Utilisées

A.1 Commandes Kali Linux – Tests d’Intrusion

Listing A.1 – Scan Nmap depuis Kali Linux vers la DMZ

```
1 # Scan des ports ouverts sur le serveur DVWA
2 nmap -sV -sC -p- 192.168.20.20
3
4 # Scan rapide de la plage DMZ
5 nmap -sn 192.168.20.0/24
```

Listing A.2 – Attaque brute-force SSH avec Hydra

```
1 # Brute-force SSH vers la machine Windows
2 hydra -l admin -P /usr/share/wordlists/rockyou.txt \
3     ssh://192.168.20.10 -t 4 -v
```

A.2 Configuration Wazuh – Intégration Shuffle

Listing A.3 – Extrait de la configuration Wazuh – Integration Shuffle (ossec.conf)

```
1 <integration>
2   <name>shuffle</name>
3   <hook_url>http://192.168.10.30:3001/api/v1/hooks/...</
   hook_url>
4   <level>7</level>
5   <alert_format>json</alert_format>
6 </integration>
```